Exp : 1 Implement Ddl And Dml Operations

Date : (Create, Insert, Update, Delete)

Aim

To implement DDL and DML operations such as CREATE, INSERT, UPDATE, and DELETE using SQL commands.

Algorithm

1. Start the program.
2. Create a database table using the CREATE command.
3. Insert records into the table using the INSERT command.
4. Display the table contents.
5. Modify existing records using the UPDATE command.
6. Delete selected records using the DELETE command.
7. Display the updated table.
8. Stop the program.

Program

```
-- DDL Operation: CREATE
CREATE TABLE Student (
  RollNo INT PRIMARY KEY,
  Name VARCHAR(50),
  Department VARCHAR(30),
  Marks INT
);

-- DML Operation: INSERT
INSERT INTO Student VALUES (1, 'Arun', 'CSE', 85);
INSERT INTO Student VALUES (2, 'Divya', 'IT', 90);
INSERT INTO Student VALUES (3, 'Karthik', 'ECE', 78);
```

```
-- Display Table
SELECT * FROM Student;

-- DML Operation: UPDATE
UPDATE Student SET Marks = 88 WHERE RollNo = 3;

-- DML Operation: DELETE
DELETE FROM Student WHERE RollNo = 1;

-- Display After UPDATE and DELETE
SELECT * FROM Student;
```

Output

Output After Insert

RollNo	Name	Department	Marks
1	Arun	CSE	85
2	Divya	IT	90
3	Karthik	ECE	78

After Update

RollNo	Name	Department	Marks
1	Arun	CSE	85
2	Divya	IT	90
3	Karthik	ECE	88

After Delete

RollNo	Name	Department	Marks
2	Divya	IT	90
3	Karthik	ECE	88

Result

Thus, the DDL and DML operations (CREATE, INSERT, UPDATE, and DELETE) were successfully implemented using SQL, and the records were created, modified, and deleted as required.

Exp : 2 Write Sql Queries Using Where, Order By,
Date : Between, In, Like Clauses

Aim

To write and execute SQL queries using WHERE, ORDER BY, BETWEEN, IN, and LIKE clauses to filter and sort data from a database table.

Algorithm

1. Create a table with required fields.
2. Insert sample records into the table.
3. Use the WHERE clause to filter records based on conditions.
4. Use ORDER BY to sort the result.
5. Use BETWEEN to select values within a range.
6. Use IN to match values from a given list.
7. Use LIKE to search patterns in text fields.
8. Observe and verify the output.

Program

```
-- Create table
CREATE TABLE Student (
  RollNo INT,
  Name VARCHAR(50),
  Department VARCHAR(30),
  Marks INT
);

-- Insert values
INSERT INTO Student VALUES (1,'Anu','CSE',85),(2,'Bala','ECE',72),
  (3,'Charan','CSE',90),(4,'Divya','EEE',65),(5,'Arun','IT',78);

-- 1. WHERE clause
SELECT * FROM Student WHERE Department = 'CSE';
```

```
-- 2. ORDER BY clause
SELECT * FROM Student ORDER BY Marks DESC;

-- 3. BETWEEN clause
SELECT * FROM Student WHERE Marks BETWEEN 70 AND 85;

-- 4. IN clause
SELECT * FROM Student WHERE Department IN ('CSE', 'IT');

-- 5. LIKE clause
SELECT * FROM Student WHERE Name LIKE 'A%';
```

Output

RollNo	Name	Department	Marks
1	Anu	CSE	85
3	Charan	CSE	90
3	Charan	CSE	90
1	Anu	CSE	85
5	Arun	IT	78
2	Bala	ECE	72
4	Divya	EEE	65

Result

Thus, SQL queries using WHERE, ORDER BY, BETWEEN, IN, and LIKE clauses were successfully written and executed, and the required results were obtained.

Exp : 3 Implement Different Types Of Joins

Date : (Inner, Outer, Left, Right, Self Join)

Aim

To implement and execute different types of SQL joins such as INNER JOIN, LEFT JOIN, RIGHT JOIN, OUTER JOIN, and SELF JOIN to combine data from multiple tables.

Algorithm

1. Create two tables with related fields.
2. Insert sample records into the tables.
3. Use INNER JOIN to retrieve matching records from both tables.
4. Use LEFT JOIN to retrieve all records from the left table and matching records from the right table.
5. Use RIGHT JOIN to retrieve all records from the right table and matching records from the left table.
6. Use OUTER JOIN to retrieve all records from both tables.
7. Use SELF JOIN to join a table with itself.
8. Observe and verify the output.

Program

```
-- Create tables
CREATE TABLE Department ( DeptID INT, DeptName VARCHAR(30) );
CREATE TABLE Employee (
  EmpID INT, EmpName VARCHAR(30), DeptID INT, ManagerID INT
);

-- Insert values
INSERT INTO Department VALUES (1,'CSE'),(2,'ECE'),(3,'EEE');
INSERT INTO Employee VALUES (101,'Anu',1,NULL),(102,'Bala',2,101),
  (103,'Charan',1,101),(104,'Divya',3,102);
```

```
-- INNER JOIN
SELECT Employee.EmpName, Department.DeptName FROM Employee
  INNER JOIN Department ON Employee.DeptID = Department.DeptID;

-- LEFT JOIN
SELECT Employee.EmpName, Department.DeptName FROM Employee
  LEFT JOIN Department ON Employee.DeptID = Department.DeptID;

-- RIGHT JOIN
SELECT Employee.EmpName, Department.DeptName FROM Employee
  RIGHT JOIN Department ON Employee.DeptID = Department.DeptID;

-- SELF JOIN
SELECT E1.EmpName AS Employee, E2.EmpName AS Manager
  FROM Employee E1 LEFT JOIN Employee E2 ON E1.ManagerID =
E2.EmpID;
```

Output

EmpName	DeptName
Anu	CSE
Bala	ECE
Charan	CSE
Divya	EEE

Result

Thus, different types of SQL joins were successfully implemented and executed, and the required outputs were obtained.

Exp : 4 Use Aggregate Functions (Count, Sum, Avg,
Date : Max, Min) With Group By And Having Clauses

Aim

To use aggregate functions COUNT, SUM, AVG, MAX, and MIN along with GROUP BY and HAVING clauses to perform calculations on grouped data.

Algorithm

1. Create a table with numeric fields.
2. Insert sample records into the table.
3. Use COUNT to count the number of records.
4. Use SUM to calculate total values.
5. Use AVG to calculate average values.
6. Use MAX and MIN to find highest and lowest values.
7. Use GROUP BY to group records.
8. Use HAVING to filter grouped data.
9. Observe and verify the output.

Program

```
-- Create table
CREATE TABLE Student (
  RollNo INT, Name VARCHAR(30), Department VARCHAR(20), Marks INT
);

-- Insert values
INSERT INTO Student VALUES (1,'Anu','CSE',85),(2,'Bala','ECE',72),
  (3,'Charan','CSE',90),(4,'Divya','EEE',65),(5,'Arun','CSE',78);

-- COUNT
SELECT Department, COUNT(*) FROM Student GROUP BY Department;

-- SUM
```

```
SELECT Department, SUM(Marks) FROM Student GROUP BY Department;

-- AVG
SELECT Department, AVG(Marks) FROM Student GROUP BY Department;

-- MAX and MIN
SELECT Department, MAX(Marks), MIN(Marks) FROM Student GROUP BY
Department;

-- HAVING
SELECT Department, AVG(Marks) FROM Student
  GROUP BY Department HAVING AVG(Marks) > 75;
```

Output

Department	COUNT(*)	SUM(Marks)	AVG(Marks)	MAX(Marks)	MIN(Marks)
CSE	3	253	84.33	90	78
ECE	1	72	72.00	72	72
EEE	1	65	65.00	65	65

Result

Thus, aggregate functions using GROUP BY and HAVING clauses were successfully executed and the required results were obtained.

Exp : 5 **Design An Er Diagram And Convert It To A**

Date : **Relational Schema Implement The Schema In Sql**

Aim

To design an ER diagram for a College Management System, convert it into a Relational Schema, and implement the schema using SQL commands.

Algorithm

1. Step 1: Identify Entities - Department, Student, Course, Enrollment.
2. Step 2: Assign primary keys and add necessary attributes.
3. Step 3: Identify foreign keys for relationships.
4. Step 4: One Department has Many Students and Many Courses.
5. Step 5: Many Students to Many Courses resolved using Enrollment.
6. Step 6: Convert ER to Relational Schema with Primary and Foreign Keys.
7. Step 7: Write CREATE TABLE statements and insert sample data.
8. Step 8: Retrieve data using SELECT queries.

Program

```
-- Create Department Table
CREATE TABLE Department ( Dept_ID INT PRIMARY KEY, Dept_Name
VARCHAR(100) );

-- Create Student Table
CREATE TABLE Student (
  Student_ID INT PRIMARY KEY, Name VARCHAR(100),
  Email VARCHAR(100), Dept_ID INT,
  FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);

-- Create Course Table
CREATE TABLE Course (
  Course_ID INT PRIMARY KEY, Course_Name VARCHAR(100),
```

```
  Credits INT, Dept_ID INT,
  FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);

-- Create Enrollment Table
CREATE TABLE Enrollment (
  Enrollment_ID INT PRIMARY KEY, Student_ID INT, Course_ID INT,
  Semester VARCHAR(20), Grade VARCHAR(5),
  FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
  FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);

-- Insert Sample Data
INSERT INTO Department VALUES (1, 'Computer Science');
INSERT INTO Student VALUES (101,'Arun','arun@gmail.com',1);
INSERT INTO Student VALUES (102,'Priya','priya@gmail.com',1);
INSERT INTO Course VALUES (201,'DBMS',4,1);
INSERT INTO Course VALUES (202,'Python',3,1);
INSERT INTO Enrollment VALUES (1,101,201,'Semester 1','A');
INSERT INTO Enrollment VALUES (2,102,202,'Semester 1','B');

-- Display Data
SELECT * FROM Student;
SELECT * FROM Course;
SELECT * FROM Enrollment;
```

Output

Result

Thus, the ER diagram was designed, converted into a relational schema, and successfully implemented using SQL commands.

Exp : 6 Apply Normalization (Up To 3nf) To A
Date : Given Dataset With Redundant Data

Aim

To apply normalization (up to Third Normal Form - 3NF) to a dataset containing redundant data and convert it into well-structured relational tables to eliminate redundancy and dependency issues.

Algorithm

1. Start with the unnormalized dataset (UNF).
2. Convert the table into 1NF by removing repeating groups and ensuring atomic values.
3. Identify the primary key.
4. Check for partial dependencies and decompose tables to achieve 2NF.
5. Identify transitive dependencies.
6. Separate attributes causing transitive dependency into new tables.
7. Establish foreign key relationships between tables.
8. The resulting tables represent the database in 3NF.

Program

```
-- Student table
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  StudentName VARCHAR(50), StudentAddress VARCHAR(50)
);
INSERT INTO Student VALUES
  (101,'Amit','Mumbai'),(102,'Neha','Delhi'),
  (103,'Rahul','Pune'),(104,'Priya','Mumbai'),(105,'Karan','Delhi');

-- Instructor table
CREATE TABLE Instructor (
```

```
  InstructorID INT PRIMARY KEY,
  InstructorName VARCHAR(50), InstructorPhone VARCHAR(15)
);
INSERT INTO Instructor VALUES
  (1,'Dr. Rao','9876543210'),(2,'Dr. Sharma','9876501234'),
  (3,'Dr. Mehta','9876512345'),(4,'Dr. Patel','9876598765');

-- Course Table
CREATE TABLE Course (
  CourseID INT PRIMARY KEY, CourseName VARCHAR(50),
  InstructorID INT, Department VARCHAR(50),
  FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID)
);
INSERT INTO Course VALUES
  (201,'DBMS',1,'Computer'),(202,'Operating System',2,'Computer'),
  (203,'Computer Networks',3,'IT'),(204,'Data Structures',4,'Computer');

-- Enrollment Table
CREATE TABLE Enrollment (
  StudentID INT, CourseID INT,
  FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
  FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
INSERT INTO Enrollment VALUES
  (101,201),(101,202),(102,201),(103,203),(104,204),(105,203);
```

Output

Result

Thus, normalization up to 3NF was successfully applied, eliminating redundancy and dependency issues from the dataset.

Exp : 7 Create And Manipulate Views;

Date : Demonstrate Indexes

Aim

To create and manipulate views in a database and to demonstrate the creation and use of indexes for efficient data retrieval.

Algorithm

1. Create Student table.
2. Insert sample records.
3. Create a view and display data.
4. Drop and recreate the view with changes.
5. Create indexes on required columns.
6. Display indexes.
7. Drop the index.

Program

```
-- Step 1: Create table
CREATE TABLE Se2 (
  ROLLNO NUMBER PRIMARY KEY, NAME VARCHAR2(50),
  DEPARTMENT VARCHAR2(50), MARKS NUMBER
);

-- Step 2: Insert data
INSERT INTO Se2 VALUES (1,'ANU','CSE',85);
INSERT INTO Se2 VALUES (2,'BALA','ECE',78);
INSERT INTO Se2 VALUES (3,'CHITRA','CSE',92);
INSERT INTO Se2 VALUES (4,'DAVID','EEE',74);

-- Step 3: Commit
COMMIT;

-- Step 4: Create view
```

```
CREATE VIEW CSE_VIEW AS
  SELECT NAME, MARKS FROM Se2 WHERE DEPARTMENT = 'CSE';

-- Step 5: Display view
SELECT * FROM CSE_VIEW;

-- Step 6: Drop view
DROP VIEW CSE_VIEW;

-- Step 7: Create modified view
CREATE VIEW CSE_VIEW AS
  SELECT NAME, MARKS FROM Se2 WHERE MARKS > 80;

-- Step 8: Display modified view
SELECT * FROM CSE_VIEW;

-- Step 9: Create index
CREATE INDEX NAME ON Se2(NAME);

-- Step 10: Display indexes
SELECT INDEX_NAME FROM USER_INDEXES WHERE TABLE_NAME
= 'STUDENT2';
```

Output

Result

Thus, views and indexes were successfully created, manipulated, and demonstrated.

Exp : 8 Simulate Transaction And Demonstrate
Date : Acid Properties

Aim

To simulate database transactions and demonstrate the ACID properties (Atomicity, Consistency, Isolation, Durability) using a simple program.

Algorithm

1. Start the transaction.
2. Read the account balance from the database.
3. Perform a transaction operation (transfer amount from one account to another).
4. Update the balance in the database.
5. If all operations are successful, commit the transaction.
6. If any error occurs, rollback the transaction.
7. Display the final balances.
8. Verify the ACID properties during the transaction execution.

Program

```
-- Create table
CREATE TABLE BankAccount (
  AccountID INT, Name VARCHAR(20), Balance INT
);

-- Insert values
INSERT INTO BankAccount VALUES (1,'Ram',5000);
INSERT INTO BankAccount VALUES (2,'Ravi',3000);

-- Deduct money from Ram
UPDATE BankAccount SET Balance = Balance - 500 WHERE AccountID = 1;

-- Add money to Ravi
```

```
UPDATE BankAccount SET Balance = Balance + 500 WHERE AccountID =
2;

-- Commit transaction
COMMIT;

-- Display table
SELECT * FROM BankAccount;
```

Output

AccountId	Name	Balance
1	Ram	4500
2	Ravi	3500

Demonstration Of ACID Properties

Atomicity: All operations happen completely or none happen. If money is deducted from Ram but not added to Ravi, the system rolls back the transaction.

Consistency: Database moves from one valid state to another. Total balance before: 5000 + 3000 = 8000. After: 4500 + 3500 = 8000.

Isolation: Multiple transactions executed at the same time do not affect each other. Each transaction works independently.

Durability: Once the transaction is committed, the changes are permanently stored in the database even if the system crashes.

Result

Thus, the transaction simulation was successfully performed and the ACID properties (Atomicity, Consistency, Isolation, Durability) were demonstrated.

Exp : 9 Demonstrate Concurrency Using
Date : Lock-Based Mechanisms

Aim

To demonstrate concurrency control using lock-based mechanisms in SQL by applying row-level locking with SELECT ... FOR UPDATE.

Algorithm

1. Create table and insert a record.
2. Start Transaction 1 and lock row using SELECT FOR UPDATE.
3. Update the balance.
4. Start Transaction 2 and try to access same row (waits).
5. Commit Transaction 1 (lock released).
6. Transaction 2 continues and updates.
7. Commit and display final result.

Program

```
CREATE TABLE bank_lock (
  id INT PRIMARY KEY, name VARCHAR(20), balance INT
);
INSERT INTO bank_lock VALUES (1,'Ram',5000);

-- Transaction 1: Lock row
SELECT * FROM bank_lock WHERE id = 1 FOR UPDATE;
UPDATE bank_lock SET balance = balance - 500 WHERE id = 1;

-- Transaction 2: Try to access same row (waits)
SELECT * FROM bank_lock WHERE id = 1 FOR UPDATE;
COMMIT; -- Transaction 1 commits, lock released

UPDATE bank_lock SET balance = balance + 1000 WHERE id = 1;
COMMIT;
```

SELECT * FROM bank_lock;

Output

Id	Name	Balance
1	Ram	5000
1	Ram	4500
1	Ram	5500

Result

Thus, concurrency using lock-based mechanisms was demonstrated successfully using SELECT ... FOR UPDATE, where one transaction waits until the locked row is released by another transaction.

Exp : 10 Create And Execute Pl/Sql Procedures,

Date : Functions, And Triggers

Aim

To create and execute PL/SQL procedures, functions, and triggers.

Algorithm

1. Start the program.
2. Enable server output using SET SERVEROUTPUT ON.
3. Create a table to store student details.
4. Insert sample records into the table.
5. Create a PL/SQL procedure to display a message.
6. Execute the procedure.
7. Create a PL/SQL function to count total number of students.
8. Execute the function and display the result.
9. Create a trigger that executes before inserting a record.
10. Insert a new record to fire the trigger.
11. Display the output.
12. Stop the program.

Program

```
SET SERVEROUTPUT ON;

-- Create Table
CREATE TABLE Students (
  Student_Id NUMBER, Student_Name VARCHAR2(50), Marks NUMBER
);

-- Insert Data
INSERT INTO Students VALUES (1,'Arun',80);
INSERT INTO Students VALUES (2,'Divya',90);
```

```
COMMIT;

-- Create Procedure
CREATE OR REPLACE PROCEDURE display_message AS
BEGIN
  DBMS_OUTPUT.PUT_LINE('Procedure executed successfully');
END;
/

-- Execute Procedure
BEGIN display_message; END;
/

-- Create Function
CREATE OR REPLACE FUNCTION get_student_count RETURN NUMBER
IS
  Total_Students NUMBER;
BEGIN
  SELECT COUNT(*) INTO Total_Students FROM Students;
  RETURN Total_Students;
END;
/

-- Execute Function
DECLARE total NUMBER;
BEGIN
  total := get_student_count;
  DBMS_OUTPUT.PUT_LINE('Total Students: ' || total);
END;
/

-- Create Trigger
CREATE OR REPLACE TRIGGER student_trigger
  BEFORE INSERT ON Students
```

```
  FOR EACH ROW
BEGIN
  DBMS_OUTPUT.PUT_LINE('Trigger executed before inserting a record');
END;
/

-- Fire Trigger
INSERT INTO Students VALUES (3,'Naveen',95);
```

Output

Procedure executed successfully

Total Students: 2

Trigger executed before inserting a record

Result

Thus, PL/SQL procedures, functions, and triggers were successfully created and executed.

Exp : 11(A)

Date :

Case Study: Library Database Schema

Aim

To design and implement a simple Library Database System using SQL to manage books, members, and book issue details.

Algorithm

1. Create tables for Books, Members, and Issue.
2. Define primary keys and foreign keys.
3. Insert sample data into Books and Members tables.
4. Insert issue record using TO_DATE() for date format.
5. Retrieve data using SELECT query.

Program

```
CREATE TABLE BooksTbl (
  Book_ID INT PRIMARY KEY, Title VARCHAR2(50),
  Author VARCHAR2(50), Quantity INT
);

CREATE TABLE MembersTbl (
  Member_ID INT PRIMARY KEY, Name VARCHAR2(50)
);

CREATE TABLE IssueTbl (
  Issue_ID INT PRIMARY KEY, Book_ID INT, Member_ID INT, Issue_Date
DATE,
  FOREIGN KEY (Book_ID) REFERENCES BooksTbl(Book_ID),
  FOREIGN KEY (Member_ID) REFERENCES MembersTbl(Member_ID)
);

INSERT INTO BooksTbl VALUES (1,'Harry Potter','J.K. Rowling',5);
INSERT INTO MembersTbl VALUES (101,'Suriya');
```

```
INSERT INTO IssueTbl (Issue_ID,Book_ID,Member_ID,Issue_Date)
  VALUES (1001,1,101,TO_DATE('2026-04-11','YYYY-MM-DD'));

SELECT * FROM BooksTbl;
SELECT * FROM MembersTbl;
SELECT * FROM IssueTbl;
```

Output

Book_id	Title	Author	Quantity
1	Harry Potter	J.K. Rowling	5

Member_id	Name
101	Suriya

Issue_id	Book_id	Member_id	Issue_date
1001	1	101	11-APR-26

Result

The Library Database was successfully created, and records for books, members, and issued books were inserted and displayed correctly.

Exp : 11(B)

Date :

Case Study: Hospital Database Schema

Aim

To create a hospital database to manage patients, doctors, and appointments.

Algorithm

1. Create Patients, Doctors, and Appointment tables.
2. Define primary and foreign keys.
3. Insert sample data.
4. Use TO_DATE() for date format.
5. Display records.

Program

```
CREATE TABLE PatientsTbl (
  Patient_ID INT PRIMARY KEY, Name VARCHAR2(50), Disease
VARCHAR2(50)
);

CREATE TABLE DoctorsTbl (
  Doctor_ID INT PRIMARY KEY, Name VARCHAR2(50), Specialization
VARCHAR2(50)
);

CREATE TABLE AppointmentTbl (
  App_ID INT PRIMARY KEY, Patient_ID INT, Doctor_ID INT, App_Date
DATE,
  FOREIGN KEY (Patient_ID) REFERENCES PatientsTbl(Patient_ID),
  FOREIGN KEY (Doctor_ID) REFERENCES DoctorsTbl(Doctor_ID)
);

INSERT INTO PatientsTbl VALUES (1,'Ravi','Fever');
INSERT INTO DoctorsTbl VALUES (101,'Dr. Kumar','General');
```

```
INSERT INTO AppointmentTbl (App_ID,Patient_ID,Doctor_ID,App_Date)
  VALUES (1001,1,101,TO_DATE('2026-04-11','YYYY-MM-DD'));

SELECT * FROM PatientsTbl;
SELECT * FROM DoctorsTbl;
SELECT * FROM AppointmentTbl;
```

Output

Patient_id	Name	Disease
1	Ravi	Fever

Doctor_id	Name	Specialization
101	Dr. Kumar	General

App_id	Patient_id	Doctor_id	App_date
1001	1	101	11-APR-26

Result

Thus, the Hospital database was created and records were inserted and displayed successfully.

Exp : 11(C)

Date :

Case Study: E-Commerce Database Schema

Aim

To create an e-commerce database to manage customers, products, and orders.

Algorithm

1. Create Customers, Products, and Orders tables.
2. Define primary and foreign keys.
3. Insert sample data.
4. Use TO_DATE() for date format.
5. Display records.

Program

```
CREATE TABLE CustomersTbl (
  Customer_ID INT PRIMARY KEY, Name VARCHAR2(50), City
VARCHAR2(50)
);

CREATE TABLE ProductsTbl (
  Product_ID INT PRIMARY KEY, Product_Name VARCHAR2(50), Price
INT
);

CREATE TABLE OrdersTbl (
  Order_ID INT PRIMARY KEY, Customer_ID INT, Product_ID INT,
Order_Date DATE,
  FOREIGN KEY (Customer_ID) REFERENCES
CustomersTbl(Customer_ID),
  FOREIGN KEY (Product_ID) REFERENCES ProductsTbl(Product_ID)
);

INSERT INTO CustomersTbl VALUES (1,'Ram','Chennai');
```

```
INSERT INTO ProductsTbl VALUES (101,'Laptop',50000);
INSERT INTO OrdersTbl (Order_ID,Customer_ID,Product_ID,Order_Date)
  VALUES (1001,1,101,TO_DATE('2026-04-11','YYYY-MM-DD'));

SELECT * FROM CustomersTbl;
SELECT * FROM ProductsTbl;
SELECT * FROM OrdersTbl;
```

Output

Customer_id	Name	City
1	Ram	Chennai

Product_id	Product_name	Price
101	Laptop	50000

Order_id	Customer_id	Product_id	Order_date
1001	1	101	11-APR-26

Result

Thus, the E-commerce database was created and records were inserted and displayed successfully.

Exp : 12

Date :

Comparison Of Relational And NoSQL Models

Aim

To compare Relational Database Model and NoSQL Model by implementing a Student Management System using SQL.

Algorithm

1. Step 1: Start SQL*Plus and connect to the database.
2. Step 2: Create tables for Student, Course, and Enrollment.
3. Step 3: Insert sample data into the tables.
4. Step 4: Retrieve data using SELECT query with joins.
5. Step 5: Represent the same data using NoSQL (document model).
6. Step 6: Compare both models based on structure and usage.

Program

```
-- Create Tables
CREATE TABLE Student_Info (
  student_id NUMBER PRIMARY KEY, name VARCHAR2(100),
  age NUMBER, email VARCHAR2(100)
);

CREATE TABLE Course_Info (
  course_id NUMBER PRIMARY KEY, course_name VARCHAR2(100)
);

CREATE TABLE Enrollment_Info (
  enrollment_id NUMBER PRIMARY KEY, student_id NUMBER, course_id
NUMBER,
  FOREIGN KEY (student_id) REFERENCES Student_Info(student_id),
  FOREIGN KEY (course_id) REFERENCES Course_Info(course_id)
);
```

```
-- Insert Data
INSERT INTO Student_Info VALUES (1,'Rahul',20,'rahul@email.com');
INSERT INTO Student_Info VALUES (2,'Priya',22,'priya@email.com');
INSERT INTO Course_Info VALUES (101,'Database Systems');
INSERT INTO Course_Info VALUES (102,'AI');
INSERT INTO Enrollment_Info VALUES (1,1,101);
INSERT INTO Enrollment_Info VALUES (2,1,102);
INSERT INTO Enrollment_Info VALUES (3,2,101);
COMMIT;

SET LINESIZE 200;
SELECT s.name, c.course_name
  FROM Student_Info s, Course_Info c, Enrollment_Info e
  WHERE s.student_id = e.student_id AND c.course_id = e.course_id;

-- NoSQL Representation (Document Model)
{ name: "Rahul", age: 20, email: "rahul@email.com",
  courses: ["Database Systems", "AI"] }
{ name: "Priya", age: 22, email: "priya@email.com",
  courses: ["Database Systems"] }
```

Output

SQL*Plus Output

Name	Course_Name
Rahul	Database Systems
Rahul	AI
Priya	Database Systems

NoSQL Model Output (MongoDB)

Name	Age	Email	Courses
Rahul	20	rahul@email.com	Database Systems, AI

Priya	22	priya@email.com	Database Systems

Difference Between Relational And NoSQL Models

Relational Model	NoSQL Model
Data stored in tables	Data stored in documents
Fixed schema	Flexible schema
Uses rows and columns	Uses JSON format
Uses primary & foreign keys	No keys, uses embedded data
JOIN is required	No JOIN required
Multiple tables	Single collection

Result

The Student Management System was successfully implemented using the relational model in SQL. The NoSQL model was represented using document format, showing flexible data storage.

Exp : 13

Date :

Mini Project: Complete Database Design

Aim

To design a simple Library Database system using SQL to manage books and members, and to perform operations like issuing and returning books.

Algorithm

1. Start the program.
2. Create a table Books with attributes (id, name, copies).
3. Create a table Members with attributes (id, name).
4. Insert sample data into both tables.
5. Issue a book by reducing the number of copies.
6. Return a book by increasing the number of copies.
7. Display all records from Books and Members tables.
8. Stop the program.

Program

```
-- Create Tables
CREATE TABLE Books (
  id INT PRIMARY KEY, name VARCHAR(30), copies INT
);

CREATE TABLE Members (
  id INT PRIMARY KEY, name VARCHAR(30)
);

-- Insert Data
INSERT INTO Books VALUES (1,'DBMS',5);
INSERT INTO Members VALUES (1,'Ram');

-- Issue Book
```

```
UPDATE Books SET copies = copies - 1 WHERE id = 1;

-- Return Book
UPDATE Books SET copies = copies + 1 WHERE id = 1;

-- Display
SELECT * FROM Books;
SELECT * FROM Members;
```

Output

Id	Name	Copies
1	DBMS	5

Id	Name
1	Ram

Result

Thus, a simple Library Management System database was successfully created, and operations like book issue and return were performed using SQL queries.

MINI PROJECT – I
E-COMMERCE SYSTEM

1. Introduction

The E-Commerce System is a relational database project designed to manage online shopping operations efficiently. It stores and processes data related to users, products, orders, order items, and payments.

The system demonstrates how structured databases can be used to handle real-world applications using SQL operations such as table creation, insertion, and multi-table JOIN queries.

2. Objectives

- To design a relational database for an e-commerce system
- To maintain structured data using tables
- To establish relationships using primary and foreign keys
- To retrieve meaningful information using SQL queries
- To implement JOIN operations for combined data analysis

3. Tools Used

- MySQL Workbench
- SQL (Structured Query Language)

4. System Design

The system consists of multiple tables connected using relationships.

Entities:

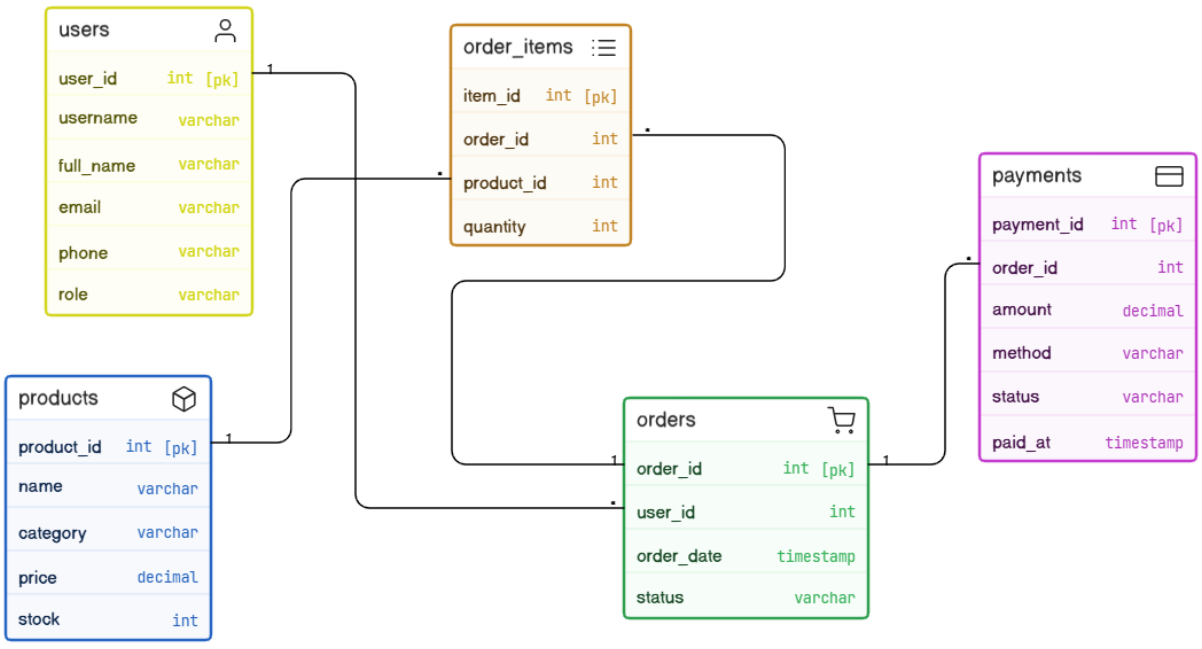
- Users
- Products
- Orders
- Order Items
- Payments

Relationships:

- One user → many orders
- One order → many products
- One product → many order items
- One order → one payment

5. ER Diagram

Description:
The diagram shows all entities and relationships. Each table has a primary key. Foreign keys establish relationships between tables. The structure follows normalized relational design.



6. Database Schema (SQL Code)

```
-- ========================
-- E-COMMERCE SYSTEM
-- ========================

CREATE DATABASE ecommerce_db;
USE ecommerce_db;


-- ========================
-- USERS TABLE
-- ========================

CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50),
  full_name VARCHAR(100),
  email VARCHAR(100),
  phone VARCHAR(20),
  role VARCHAR(20)
);


-- ========================
-- PRODUCTS TABLE
-- ========================

CREATE TABLE products (
  product_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  category VARCHAR(50),
  price DECIMAL(8,2),
  stock INT
);
```

```sql
-- ========================
-- ORDERS TABLE
-- ========================

CREATE TABLE orders (
  order_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  order_date TIMESTAMP,
  status VARCHAR(20),
  FOREIGN KEY (user_id) REFERENCES users(user_id)
);


-- ========================
-- ORDER ITEMS TABLE
-- ========================

CREATE TABLE order_items (
  item_id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT,
  product_id INT,
  quantity INT,
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- ========================
-- PAYMENTS TABLE
-- ========================

CREATE TABLE payments (
  payment_id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT,
  amount DECIMAL(8,2),
  method VARCHAR(20),
  status VARCHAR(20),
  paid_at TIMESTAMP,
  FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

7. Insert Values

```
-- ========================
-- INSERT USERS
-- ========================
INSERT INTO users (username, full_name, email, phone, role) VALUES
('Arun','Arun Kumar','arunkumar@gmail.com','9876543210','customer'),
('Divya','Divya Nair','divya@gmail.com','9876501234','customer'),
('Admin','Admin User','admin@gmail.com','9999999999','admin');


-- ========================
-- INSERT PRODUCTS
-- ========================
INSERT INTO products (name, category, price, stock) VALUES
('Laptop','Electronics',50000.00,10),
('Mobile','Electronics',20000.00,20),
('Headphones','Accessories',1500.00,50),
('Keyboard','Accessories',800.00,30);


-- ========================
-- INSERT ORDERS
-- ========================
INSERT INTO orders (user_id, order_date, status) VALUES
(1,'2026-05-01 10:30:00','placed'),
(2,'2026-05-02 12:00:00','placed');


-- ========================
-- INSERT ORDER ITEMS
-- ========================
INSERT INTO order_items (order_id, product_id, quantity) VALUES
(1,1,1),
(1,3,2),
(2,2,1);
```

```
-- ========================
-- INSERT PAYMENTS
-- ========================
INSERT INTO payments (order_id, amount, method, status, paid_at)
VALUES
(1,53000.00,'card','success','2026-05-01 11:00:00'),
(2,20000.00,'upi','success','2026-05-02 12:30:00');
```

8. Sample Queries

8.1 Display All Users

```
SELECT * FROM users;
```

ID	Username	Full Name	Email	Phone	Role
1	Arun	Arun Kumar	arunkumar@gmail.com	9876543210	customer
2	Divya	Divya Nair	divya@gmail.com	9876501234	customer
3	Admin	Admin User	admin@gmail.com	9999999999	admin

8.2 Display All Products

```
SELECT * FROM products;
```

ID	Name	Category	Price	Stock
1	Laptop	Electronics	50000.00	10
2	Mobile	Electronics	20000.00	20

3	Headphones	Accessories	1500.00	50
4	Keyboard	Accessories	800.00	30

8.3 Display All Orders

```
SELECT * FROM orders;
```

ID	User ID	Date	Status
1	1	2026-05-01 10:30:00	placed
2	2	2026-05-02 12:00:00	placed

8.4 Display All Order Items

```
SELECT * FROM order_items;
```

ID	Order ID	Product ID	Quantity
1	1	1	1
2	1	3	2
3	2	2	1

8.5 Display All Payments

```
SELECT * FROM payments;
```

ID	Order ID	Amount	Method	Status	Paid At
1	1	53000.00	card	success	2026-05-01 11:00:00
2	2	20000.00	upi	success	2026-05-02 12:30:00

9. Join Query (Main Report)

```
SELECT
u.username,
o.order_id,
p.name AS product_name,
oi.quantity,
p.price,
(p.price * oi.quantity) AS item_total,
pay.amount AS order_total,
o.status,
pay.method,
pay.paid_at
FROM orders o
JOIN users u ON o.user_id = u.user_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
JOIN payments pay ON o.order_id = pay.order_id;
```

10. Report Output

User	Order	Product	Qty	Price	Item Total	Order Total	Status	Method	Time
Arun	1	Laptop	1	50000.00	50000.00	53000.00	placed	card	2026-05-01 11:00:00
Arun	1	Headphones	2	1500.00	3000.00	53000.00	placed	card	2026-05-01 11:00:00
Divya	2	Mobile	1	20000.00	20000.00	20000.00	placed	upi	2026-05-02 12:30:00

11. Conclusion

The E-Commerce System was successfully designed and implemented using MySQL. The system efficiently manages product sales data and demonstrates the use of relational databases and SQL queries, including JOIN operations, to generate meaningful reports.

MINI PROJECT – II
BUS BOOKING SYSTEM

1. Introduction

The Bus Booking System is a relational database project designed to manage transportation services efficiently. It stores and processes data related to users, buses, routes, schedules, bookings, and payments.

The system demonstrates how structured databases can be used to handle real-world applications using SQL operations such as table creation, insertion, and multi-table JOIN queries.

2. Objectives

- To design a relational database for a bus booking system
- To maintain structured data using tables
- To establish relationships using primary and foreign keys
- To retrieve meaningful information using SQL queries
- To implement JOIN operations for combined data analysis

3. Tools Used

- MySQL Workbench
- SQL (Structured Query Language)

4. System Design

The system consists of multiple tables connected using relationships.

Entities:

- Users
- Terminals
- Drivers
- Buses
- Routes
- Schedules
- Seats
- Bookings
- Payments

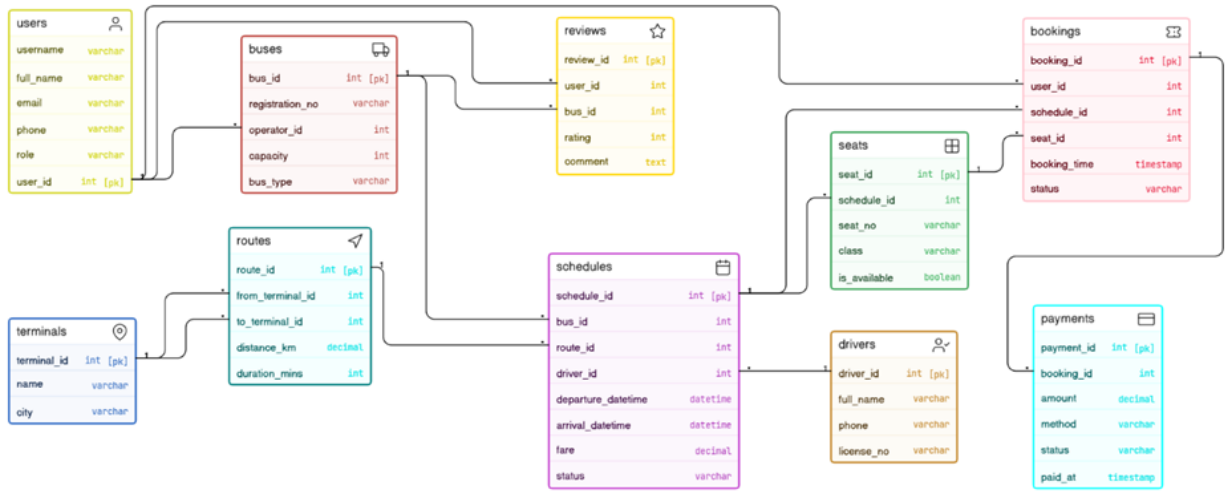
Relationships:

- One user → many bookings
- One bus → many schedules
- One route → many schedules
- One schedule → many seats
- One booking → one payment

5. ER Diagram

Description:

- The diagram shows all entities and relationships.
- Each table has a primary key.
- Foreign keys establish relationships between tables.
- The structure follows normalized relational design.



6. Database Schema (SQL Code)

```
-- ========================
-- BUS BOOKING SYSTEM
-- ========================

CREATE DATABASE bus_booking;
USE bus_booking;

-- ========================
-- USERS TABLE
-- ========================
CREATE TABLE users (
   user_id INT AUTO_INCREMENT PRIMARY KEY,
   username VARCHAR(50),
   full_name VARCHAR(100),
   email VARCHAR(100),
   phone VARCHAR(20),
   role VARCHAR(20)
);

-- ========================
-- TERMINALS TABLE
-- ========================
CREATE TABLE terminals (
   terminal_id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100),
   city VARCHAR(50)
);

-- ========================
-- DRIVERS TABLE
-- ========================
CREATE TABLE drivers (
   driver_id INT AUTO_INCREMENT PRIMARY KEY,
   full_name VARCHAR(100),
   phone VARCHAR(20),
   license_no VARCHAR(50)
);
```

```
-- =========================
-- BUSES TABLE
-- =========================

CREATE TABLE buses (
   bus_id INT AUTO_INCREMENT PRIMARY KEY,
   registration_no VARCHAR(20),
   operator_id INT,
   capacity INT,
   bus_type VARCHAR(20),
   FOREIGN KEY (operator_id) REFERENCES users(user_id)
);

-- =========================
-- ROUTES TABLE
-- =========================

CREATE TABLE routes (
   route_id INT AUTO_INCREMENT PRIMARY KEY,
   from_terminal_id INT,
   to_terminal_id INT,
   distance_km DECIMAL(6,2),
   duration_mins INT,
   FOREIGN KEY (from_terminal_id) REFERENCES terminals(terminal_id),
   FOREIGN KEY (to_terminal_id) REFERENCES terminals(terminal_id)
);

-- =========================
-- SCHEDULES TABLE
-- =========================

CREATE TABLE schedules (
   schedule_id INT AUTO_INCREMENT PRIMARY KEY,
   bus_id INT,
   route_id INT,
   driver_id INT,
   departure_datetime DATETIME,
   arrival_datetime DATETIME,
```

```
    fare DECIMAL(8,2),
    status VARCHAR(20),
    FOREIGN KEY (bus_id) REFERENCES buses(bus_id),
    FOREIGN KEY (route_id) REFERENCES routes(route_id),
    FOREIGN KEY (driver_id) REFERENCES drivers(driver_id)
);


-- =========================
-- SEATS TABLE
-- =========================

CREATE TABLE seats (
    seat_id INT AUTO_INCREMENT PRIMARY KEY,
    schedule_id INT,
    seat_no VARCHAR(10),
    class VARCHAR(20),
    is_available TINYINT(1),
    FOREIGN KEY (schedule_id) REFERENCES schedules(schedule_id)
);


-- =========================
-- BOOKINGS TABLE
-- =========================

CREATE TABLE bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    schedule_id INT,
    seat_id INT,
    booking_time DATETIME,
    status VARCHAR(20),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (schedule_id) REFERENCES schedules(schedule_id),
    FOREIGN KEY (seat_id) REFERENCES seats(seat_id)
);
```

```
-- ========================
-- PAYMENTS TABLE
-- ========================

CREATE TABLE payments (
   payment_id INT AUTO_INCREMENT PRIMARY KEY,
   booking_id INT,
   amount DECIMAL(8,2),
   method VARCHAR(20),
   status VARCHAR(20),
   paid_at DATETIME,
   FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)
);
```

7. Insert Values

```
-- ========================
-- INSERT USERS
-- ========================
INSERT INTO users (username, full_name, email, phone, role) VALUES
('arun','Arun Kumar','arun@gmail.com','9876543210','customer'),
('divya','Divya Nair','divya@gmail.com','9876501234','customer'),
('admin','Admin User','admin@gmail.com','9999999999','admin');

-- ========================
-- INSERT TERMINALS
-- ========================
INSERT INTO terminals (name, city) VALUES
('Koyambedu','Chennai'),
('Madiwala','Bangalore'),
('Gandhipuram','Coimbatore');

-- ========================
-- INSERT DRIVERS
-- ========================
INSERT INTO drivers (full_name, phone, license_no) VALUES
('Murugan','9876543211','TN12345'),
('Kumar','9876543212','KA67890');
```

```
-- ========================
-- INSERT BUSES
-- ========================
INSERT INTO buses (registration_no, operator_id, capacity, bus_type)
VALUES
('TN01AB1234',3,40,'AC'),
('KA05CD5678',3,35,'Sleeper');


-- ========================
-- INSERT ROUTES
-- ========================
INSERT INTO routes (from_terminal_id, to_terminal_id, distance_km,
duration_mins) VALUES
(1,2,350.00,360),
(2,3,300.00,300);


-- ========================
-- INSERT SCHEDULES
-- ========================
INSERT INTO schedules (bus_id, route_id, driver_id,
departure_datetime, arrival_datetime, fare, status) VALUES
(1,1,1,'2025-11-20 21:00:00','2025-11-21
05:00:00',800.00,'scheduled'),
(2,2,2,'2025-11-22 20:00:00','2025-11-23
04:00:00',900.00,'scheduled');


-- ========================
-- INSERT SEATS
-- ========================
INSERT INTO seats (schedule_id, seat_no, class, is_available) VALUES
(1,'1A','regular',1),
(1,'1B','regular',1),
(2,'2A','vip',1),
(2,'2B','vip',1);
```

```
-- ========================
-- INSERT BOOKINGS
-- ========================
INSERT INTO bookings (user_id, schedule_id, seat_id, booking_time,
status) VALUES
(1,1,1,'2026-05-02 21:16:17','booked'),
(2,2,3,'2026-05-02 21:16:17','booked');


-- ========================
-- INSERT PAYMENTS
-- ========================
INSERT INTO payments (booking_id, amount, method, status, paid_at)
VALUES
(1,800.00,'upi','success','2026-05-02 21:16:17'),
(2,900.00,'card','success','2026-05-02 21:16:17');
```

8. Sample Queries

8.1 Display All Users

```
SELECT * FROM users;
```

ID	Username	Full Name	Email	Phone	Role
1	arun	Arun Kumar	arun@gmail.com	9876543210	customer
2	divya	Divya Nair	divya@gmail.com	9876501234	customer
3	admin	Admin User	admin@gmail.com	9999999999	admin

8.2 Display All Buses

```
SELECT * FROM buses;
```

Bus ID	Registration No	Operator ID	Capacity	Bus Type
1	TN01AB1234	3	40	AC
2	KA05CD5678	3	35	Sleeper

8.3 Display All Routes

```
SELECT * FROM routes;
```

Route ID	From Terminal	To Terminal	Distance (KM)	Duration (mins)
1	1	2	350.00	360
2	2	3	300.00	300

8.4 Display All Schedules

```
SELECT * FROM schedules;
```

Schedule ID	Bus ID	Route ID	Driver ID	Departure	Arrival	Fare	Status
1	1	1	1	2025-11-20 21:00:00	2025-11-21 05:00:00	800.00	scheduled
2	2	2	2	2025-11-22 20:00:00	2025-11-23 04:00:00	900.00	scheduled

8.5 Display All Bookings

```
SELECT * FROM bookings;
```

Booking ID	User ID	Schedule ID	Seat ID	Booking Time	Status
1	1	1	1	2026-05-02 21:16:17	booked
2	2	2	3	2026-05-02 21:16:17	booked

8.6 Display All Payments

```
SELECT * FROM payments;
```

ID	Booking ID	Amount	Method	Status	Paid At
1	1	800.00	upi	success	2026-05-02 21:16:17
2	2	900.00	card	success	2026-05-02 21:16:17

9. Join Query (Main Report)

```
SELECT
   u.username,
   b.booking_id,
   bus.registration_no AS bus_no,
   bus.bus_type,
   t1.city AS from_city,
   t2.city AS to_city,
   s.seat_no,
   sch.fare,
   p.amount,
   b.booking_time
FROM bookings b
JOIN users u ON b.user_id = u.user_id
JOIN seats s ON b.seat_id = s.seat_id
JOIN schedules sch ON b.schedule_id = sch.schedule_id
JOIN buses bus ON sch.bus_id = bus.bus_id
JOIN routes r ON sch.route_id = r.route_id
JOIN terminals t1 ON r.from_terminal_id = t1.terminal_id
JOIN terminals t2 ON r.to_terminal_id = t2.terminal_id
JOIN payments p ON b.booking_id = p.booking_id;
```

10. Report Output

User	Booking	Bus No	Type	From	To	Seat	Fare	Paid	Time
arun	1	TN01AB1234	AC	Chennai	Bangalore	1A	800.00	800.00	2026-05-02 21:16:17
divya	2	KA05CD5678	Sleeper	Bangalore	Coimbatore	2A	900.00	900.00	2026-05-02 21:16:17

11. Conclusion

The Bus Booking System was successfully designed and implemented using MySQL. The system efficiently manages transportation data and demonstrates the use of relational databases and SQL queries, including JOIN operations, to generate meaningful reports.